

1 Vi phân nhiều biến và tối ưu hoá (Matrix Calculus and Optimization)

PHỤ THUỘC A/01-toan-toi-thieu/01-dai-so-tuyen-tinh (phổ trị riêng, số điều kiện — toàn bộ phần “hình dạng landscape” dưới đây là cách đọc lại phổ ấy).

TÓM TẮT

Mục này giải thích cấu trúc của phép lấy đạo hàm trên đồ thị tính toán và hình dạng của bài toán tối ưu mà huấn luyện thực sự giải. Đọc xong bạn tự derive được backward của một tầng bất kỳ mà không nhầm chiều. Bạn biết vì sao **lan truyền ngược** (*backpropagation*) đắt về bộ nhớ chứ không đắt về phép tính. Và bạn chẩn đoán được “huấn luyện chậm” thuộc loại nào trong ba loại: điều hoà tệ, learning rate sai, hay tín hiệu giám sát yếu. Nếu chỉ nhớ một điều: *khớp shape trước, dấu và hệ số sau* — và trước khi đổ lỗi cho tính không lỗi, hãy kiểm tra số điều kiện, vì phần lớn “bài toán khó tối ưu” hoá ra chỉ là bài toán bị điều hoà tệ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình và tham số** — mô hình là một hàm có tham số nhận đầu vào và trả về dự đoán; tham số là các con số bên trong hàm đó mà quá trình huấn luyện được phép thay đổi.
- **hàm mất mát (loss function)** — một con số đo mức sai của dự đoán so với nhãn. Huấn luyện là quá trình đi tìm bộ tham số làm con số này nhỏ nhất.
- **gradient** — vector các đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm loss tăng nhanh nhất, nên ta cập nhật theo hướng ngược lại.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật. Quá lớn thì nghiệm bật ra ngoài và phân kỳ; quá nhỏ thì hội tụ chậm tới mức vô dụng.
- **mini-batch và epoch** — mini-batch là một nhóm nhỏ mẫu dùng để ước lượng gradient cho một lần cập nhật; một epoch là một lượt đi hết toàn bộ tập huấn luyện.
- **tập validation** — phần dữ liệu tách riêng, không dùng để cập nhật tham số, chỉ dùng để đo xem mô hình đang tốt lên hay xấu đi. Nó tồn tại vì loss trên tập huấn luyện luôn giảm kể cả khi mô hình đang học vẹt.
- **hàm lồi (convex)** — hàm mà đoạn thẳng nối hai điểm bất kỳ trên đồ thị luôn nằm trên đồ thị — hình dạng “lồng chảo”. Với hàm lồi, mọi cực tiểu địa phương đều là cực tiểu toàn cục; mạng nơ-ron thì không lồi.
- **Jacobian và Hessian** — Jacobian là ma trận đạo hàm bậc nhất của một hàm nhiều đầu vào nhiều đầu ra — nó là bản tuyến tính hoá tốt nhất tại một điểm. Hessian là ma trận đạo hàm bậc hai, đo độ cong.

A. Đặt vấn đề

Bạn đã viết **ma trận Jacobian** (*Jacobian*) bằng tay cho một hàm chi phí **sai số tái chiếu** (*reprojection error*) và đã từng mất một buổi chiều vì nhầm dấu hoặc nhầm thứ tự nhân. Trong deep learning, **autograd** làm hộ việc đó, nên câu hỏi đổi hình. Không còn là “đạo hàm bằng bao nhiêu”, mà là “vì sao **gradient** chết ở tầng thứ ba”, “vì sao đổi thứ tự hai phép toán làm bộ nhớ tăng gấp đôi”, và “vì sao **hàm mất mát** (*loss function*) giảm nhưng mô hình không học được gì”. Trả lời những câu đó cần hiểu *cấu trúc* của phép lấy đạo hàm, không cần kỹ năng đạo hàm tay.

Học xong mục này thì làm được gì mà trước đó không làm được? Ba việc: viết và kiểm chứng một **lượt ngược** (*backward pass*) tự cài trong mười lăm phút thay vì nửa ngày; đọc một đường cong huấn luyện và đặt tên đúng cho nguyên nhân trước khi thử ngẫu nhiên các siêu tham số; và hiểu vì sao mọi optimizer hiện đại đều là biến thể của cùng một ý tưởng, nên không cần học thuộc từng cái.

B. Bộ máy vi phân: khớp shape trước

TRỰC GIÁC

Quy tắc chuỗi ở dạng ma trận là một bài toán *lắp ống dẫn*. Mỗi tầng là một cái ống có đường kính vào và đường kính ra; Jacobian của tầng là tấm chuyển tiếp kích thước (đường kính ra) $\times$ (đường kính vào). Backprop là ghép các tấm chuyển tiếp từ cuối về đầu, và cách duy nhất để ghép sai là ghép hai tấm không khớp đường kính. Vì thế: khớp shape trước, lo dấu và hệ số sau — nếu shape khớp thì công thức chỉ còn sai được ở một hằng số hoặc một dấu trừ, và cả hai lộ ra ngay trong một lần **kiểm tra gradient** (*gradient check*).

Với hàm $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, Jacobian $J \in \mathbb{R}^{m \times n}$ là bản tuyến tính hoá tốt nhất tại một điểm:

$$f(x + \delta) \approx f(x) + J\delta.$$

Nói bằng lời: *trong một lân cận đủ nhỏ, mọi hàm trơn đều là một phép nhân ma trận* — và đó là toàn bộ lý do vi phân hữu ích. **Quy tắc chuỗi** (*chain rule*) nói Jacobian của hàm hợp là tích các Jacobian theo đúng thứ tự hợp, $J_{g \circ f} = J_g J_f$.

Toàn bộ lan truyền ngược [1] chỉ là một nhận xét về *thứ tự nhân*. Đầu ra cuối là một số vô hướng, tức $m = 1$ ở tầng hàm mất mát. Khi đó nhân tích ấy từ trái sang phải rẻ hơn từ phải sang trái rất nhiều. Lý do: mỗi bước ta nhân một vector hàng với một ma trận, thay vì nhân hai ma trận. Đó là lý do nó tên là *reverse-mode automatic differentiation*, tức là đi ngược từ đầu ra vô hướng về từng tham số; cần phân biệt *lan truyền ngược* là thuật toán còn **lượt ngược** (*backward pass*) là một lần chạy nó. Đây cũng là lý do nó tốn bộ nhớ: muốn nhân ngược thì phải giữ lại giá trị trung gian của lượt xuôi. Đánh đổi bộ nhớ lấy tính toán ấy quay lại nguyên vẹn ở D dưới cái tên **gradient checkpointing** [2].

↔ **ràng buộc bởi** D — bộ nhớ kích hoạt do lượt xuôi để lại mới là thứ chặn batch size trên một GPU, không phải số trọng số

Phiên bản đơn giản hoá của mọ khớp shape. Nếu $Y = XW$ với $X \in \mathbb{R}^{B \times d_{\text{in}}}$, $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$, thì gradient của một đại lượng phải có đúng shape của chính đại lượng đó. Vậy $\partial L / \partial W$ phải là $d_{\text{in}} \times d_{\text{out}}$; hai thứ có sẵn để ghép ra kích thước đó là $X^\top$ ($d_{\text{in}} \times B$) và $\partial L / \partial Y$ ($B \times d_{\text{out}}$); chỉ có một cách ghép hợp lệ, nên

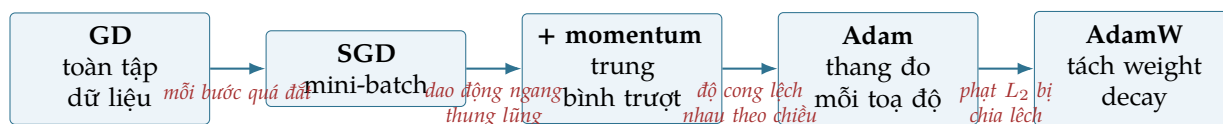
$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial Y}, \quad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^\top.$$

Không cần nhớ gì, chỉ cần đếm chiều. Quy tắc này giải quyết khoảng chín phần mười trường hợp thực tế; phần còn lại — chuẩn hoá, **softmax**, phép toán chia sẻ trọng số — thì kiểm bằng `torch.autograd.gradcheck` trên đầu vào kiểu `double`, và nếu sai thì gần như luôn sai ở một chỗ tổng-hợp-theo-*batch* bị bỏ quên.

Ma trận Hessian (*Hessian*) gần như không bao giờ được lập tường minh, vì kích thước của nó là bình phương số tham số. Nhưng *tích Hessian với một vector* tính được với chi phí cỡ một lượt ngược, và thế là đủ để đo độ cong theo một hướng. Vai trò của Hessian ở đây không phải để làm bước Newton mà để *đọc bài toán*: trị riêng lớn nhất chặn **learning rate** ổn định, phổ trị riêng cho biết landscape dẹt hay nhọn, trị riêng âm cho biết ta đang ở **điểm yên ngựa** (*saddle point*) chứ không phải cực tiểu.

C. Vì sao optimizer có hình dạng như hiện nay

Hình 1 đọc theo trục thời gian: mỗi bước sinh ra vì bước trước để lại một nút thắt cụ thể, không phải vì ai đó muốn thêm một siêu tham số.



Hình 1. Dòng tiến hoá của optimizer. Nhấn dưới mỗi mũi tên là *nút thắt* mà bước kế tiếp gỡ; đọc theo chiều này thì không cần học thuộc từng thuật toán, chỉ cần nhớ nó sinh ra để chữa cái gì.

Mạch 2 — cùng dòng tiến hoá, kèm giá phải trả

- 1 Vấn đề** hàm mất mát nhiều chiều, không có nghiệm đóng.

Giải pháp đi ngược hướng gradient từng bước nhỏ.

Giả định hàm khả vi và trơn đủ để tuyến tính hoá còn đúng trong lân cận cỡ learning rate.

Giá phải trả phải chọn learning rate; chọn sai thì hoặc phân kỳ hoặc bò.
- 2 Vấn đề** tính gradient trên toàn tập dữ liệu quá đắt.

Giải pháp ước lượng trên một mini-batch.

Hệ quả mỗi bước rẽ đi hàng nghìn lần nên ta đi được nhiều bước hơn với cùng ngân sách.

Vấn đề mới gradient thành biến ngẫu nhiên, với nhiều tỉ lệ nghịch với căn của **kích thước batch** (*batch size*). Tăng batch gấp bốn chỉ giảm nhiễu một nửa. Quan hệ lợi ích giảm dần đó quyết định phần lớn cách chọn batch size [3].
- 3 Vấn đề** các hướng có độ cong rất khác nhau nên một learning rate chung không vừa cho hướng nào.

Giải pháp co giãn từng toạ độ theo lịch sử độ lớn gradient, tức họ Adam [4].

Giả định độ cong xấp xỉ tách rời được theo từng toạ độ.

Giá phải trả mất đảm bảo hội tụ của SGD trong vài trường hợp, và sinh tương tác lạ với *weight decay* — chính chỗ đó đẻ ra AdamW [5].
- 4 Vấn đề** nhiều mini-batch làm quỹ đạo dao động ở cuối quá trình.

Giải pháp giảm learning rate theo lịch trình.

Hệ quả bước cuối đủ nhỏ để trung bình hoá nhiễu.

Vấn đề mới lịch trình thành một siêu tham số nhạy, tương tác với cả batch size lẫn số epoch — nên chép kiến trúc mà không chép lịch trình thì không tái lập được kết quả.

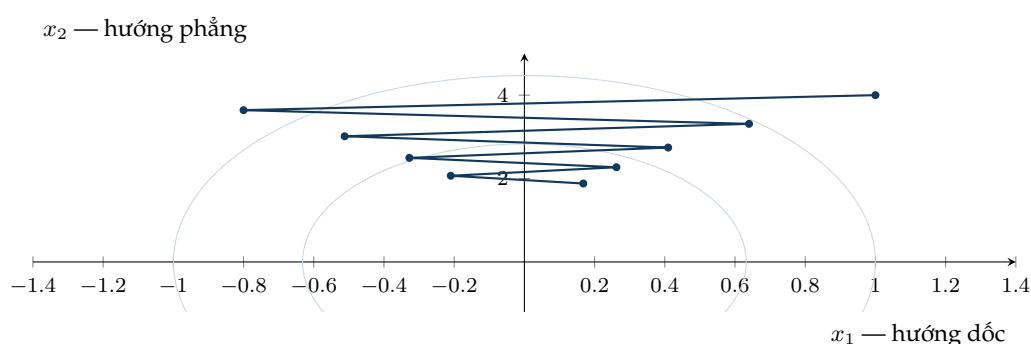
Bảng 1. Bốn optimizer thường gặp trên cùng tiêu chí. Cột cuối quan trọng nhất.

Thuật toán	Cơ chế	Mạnh khi nào	Hỏng khi nào
SGD	Bước theo gradient mini-batch	Bài toán điều hoà tốt sau chuẩn hoá; hay tổng quát hoá tốt hơn trên thị giác	Landscape lệch thang đo mạnh; cần rất nhiều công tune learning rate
SGD + momentum	Trung bình trượt gradient	Thung lũng hẹp dài — triệt tiêu dao động ngang	Momentum quá lớn gây vọt lố ở giai đoạn cuối
Adam	Chia mỗi toạ độ cho ước lượng độ lớn riêng	Gradient thưa hoặc lệch thang đo mạnh, ví dụ <i>transformer</i>	Đầu quá trình khi ước lượng phương sai chưa ổn (cần <i>warmup</i>); tổng quát hoá đôi khi kém hơn SGD
AdamW	Như Adam, tách weight decay khỏi bước thích nghi	Mặc định hợp lý cho hầu hết mô hình lớn hiện nay	Vẫn cần warmup và lịch trình; không cứu được bài toán mà tín hiệu giám sát sai

D. Lỗi hay không lỗi, và hình dạng thật của landscape

Câu nói phổ thông “deep learning khó vì hàm mất mát không lồi” đúng về chữ nhưng dẫn tới trực giác sai. Với bài toán **lồi** (*convex*), mọi cực tiểu địa phương là cực tiểu toàn cục và ta có chứng nhận dừng. Với bài toán không lồi cả hai thứ đó mất. Nhưng thực nghiệm trên mạng lớn cho thấy thủ phạm làm chậm không phải các cực tiểu địa phương tồi: trong không gian rất nhiều chiều, một điểm dừng ngẫu nhiên gần như chắc chắn là điểm yên ngựa, vì để là cực tiểu thì hàng triệu trị riêng phải dương *cùng lúc*. Cái làm chậm là vùng phẳng quanh yên ngựa và vùng có số điều kiện tệ — tức đúng hiện tượng ở mục trước.

Ví dụ nhỏ nhất tính tay được, và nó đủ giải thích cả hiện tượng. Với $L(x) = \frac{1}{2}(x_1^2 + \varepsilon x_2^2)$, hạ gradient bước η nhân thành phần thứ nhất với $(1 - \eta)$ và thành phần thứ hai với $(1 - \eta\varepsilon)$ mỗi vòng. Ổn định buộc $\eta < 2$. Lấy $\eta = 1,8$ và $\varepsilon = 0,05$. Hướng thứ nhất dao động đổi dấu mỗi bước với hệ số $-0,8$; hướng thứ hai mỗi vòng chỉ co 9%. Tỉ số $1/\varepsilon = 20$ chính là số điều kiện. Hình 2 vẽ đúng quỹ đạo đó: chi phí không nằm ở chỗ “đi xuống”, nó nằm ở chỗ đi ngang thung lũng.



Hình 2. Hạ gradient trên hàm toàn phương có số điều kiện 20: quỹ đạo này qua này lại theo hướng dốc trong khi bò rất chậm theo hướng phẳng. Chuẩn hoá đầu vào và các optimizer thích nghi đều nhắm vào việc làm hai trục của ellipse gần bằng nhau, tức là làm hình này tròn lại.

Khác biệt thứ hai, thực dụng hơn: khi không lồi, *cực tiểu nào* ta rơi vào là một phần của kết quả. Có bằng chứng thực nghiệm rằng nghiệm ở vùng dẹt tổng quát hoá tốt hơn nghiệm ở vùng nhọn, và batch lớn có xu hướng rơi vào vùng nhọn nếu không hiệu chỉnh gì [6]. Đây là quan sát đo được

và tái lập nhiều lần, nhưng cách giải thích vẫn còn tranh cãi và định nghĩa “dệt” phụ thuộc cách tham số hoá — hãy nhớ ở mức hiện tượng, đừng nhớ ở mức định lý.

E. Điều kiện dừng nào có nghĩa

Trong tối ưu cổ điển ta dừng khi chuẩn gradient nhỏ. Trong huấn luyện mạng, tiêu chí đó gần như vô dụng: gradient mini-batch không bao giờ nhỏ vì nhiễu lấy mẫu, và ngay cả gradient toàn phần nhỏ cũng không nói gì về chất lượng mô hình. Thứ thực sự được dùng là dừng theo chỉ số trên tập *validation* — tức dừng khi mục tiêu ta *quan tâm*, không phải mục tiêu ta *tối ưu*, ngừng cải thiện.

→ **tiền đề** A/01 — câu hỏi “validation đã cải thiện chưa” là câu hỏi thống kê về sai số lấy mẫu, được trả lời ở mục xác suất kế tiếp

Chuyển từ “điều kiện dừng của thuật toán” sang “điều kiện dừng của phép ước lượng ngoài mẫu” là một dịch chuyển tư duy lớn. Nó cũng là cầu nối sang A/01/03. Câu hỏi “validation đã ngừng cải thiện chưa” là câu hỏi thống kê, không phải câu hỏi giải tích.

F. Giới hạn và trường hợp hỏng

- **Giả định khả vi bị vi phạm thường xuyên hơn ta tưởng.** *ReLU* không khả vi tại không (vô hại, tập điểm có độ đo không), nhưng các phép toán rời rạc thật thì không cứu được: bước lọc bỏ các hộp trùng nhau và chỉ giữ hộp điểm cao nhất (*non-maximum suppression*) dùng *argmax*, phép lấy mẫu cứng, chỉ số nguyên trong **lượng tử hoá** (*quantization*) đều làm gradient bằng không ở mọi nơi. Cả một mảng kỹ thuật ở C và D — *surrogate loss, straight-through estimator, Gumbel-softmax* — tồn tại chỉ để vá chỗ này, và mỗi cách vá đưa vào một thiên lệch riêng.
- **hệ quả của** C — mọi cách vá chỗ không khả vi đều sinh ra từ đúng một sự thật: gradient bằng không ở mọi nơi trên một phép toán rời rạc
- **Autograd không cứu được sai số số học.** Gradient có thể đúng công thức nhưng vẫn tràn hoặc mất chính xác ở *half precision*. Triệu chứng là loss thành NaN sau vài trăm bước. Đó là lý do tồn tại của *loss scaling* trong huấn luyện hỗn hợp độ chính xác [7]. Ở chiều ngược lại, gradient check với float32 hay báo sai giả; phải kiểm ở double.
- **Đừng suy từ bài toán lỗi sang mạng sâu.** Trực giác “quỹ đạo trơn tru đi xuống lòng chảo” mô tả sai cái đang xảy ra: quỹ đạo thật đi qua vùng có độ cong đổi dấu, và learning rate lớn ở giai đoạn đầu đóng vai trò **chính quy hoá** (*regularization*) chứ không chỉ là “đi nhanh” — giảm nó quá sớm làm mất tác dụng đó.

CẠM BÃY

Loss giảm không có nghĩa mô hình đang học thứ bạn muốn. Ba trường hợp hay gặp và cách phân biệt: loss giảm vì mô hình học thiên lệch của nhãn (kiểm bằng cách so với *baseline* đoán lớp phổ biến nhất); loss train giảm mà validation đi ngang vì **rò rỉ dữ liệu** (*data leakage*) làm hai tập không độc lập (kiểm bằng cách chia theo nhóm nguồn); loss giảm đều nhưng chỉ số nghiệp vụ đứng yên vì hàm mất mát không đại diện tốt cho chỉ số đó (kiểm bằng cách vẽ tương quan giữa hai đại lượng). Luôn xem loss như một *proxy* có sai lệch; B/09 mổ xẻ sai lệch ấy.

CẦN NHỚ

Khớp shape trước, dấu và hệ số sau. Trước khi đổ lỗi cho tính không lồi, hãy kiểm tra số điều kiện. Và đừng bao giờ dừng huấn luyện theo chuẩn gradient — dừng theo chỉ số trên tập validation, vì đó mới là đại lượng bạn thật sự quan tâm. [T1]

G. Kết nối

Mục này nhận trực tiếp từ A/01/01-dai-so-tuyen-tinh (Hessian và phổ của nó) và chuyển tiếp sang A/01/03-xac-suot-va-thong-tin, nơi câu hỏi “validation ngừng cải thiện chưa” được trả lời bằng công cụ thống kê thay vì bằng mắt. Theo chiều ngang, đây là tiền đề của hai chỗ. Thứ nhất là toàn bộ chương optimizer và lịch trình learning rate ở D. Thứ hai là phần thiết kế hàm mất mát ở B/09, nơi ta thấy nhiều lựa chọn loss thật ra là lựa chọn về *hình dạng gradient* chứ không về ý nghĩa thống kê.

Câu hỏi tự kiểm tra (trả lời không nhìn bài)

1. Vì sao chế độ ngược tồn bộ nhớ còn chế độ xuôi thì không, và điều đó dẫn tới kỹ thuật gì khi mô hình quá lớn?
2. Bạn derive được $\partial L / \partial W$ mà không cần nhớ công thức. Lập luận đó dựa trên tính chất nào, và khi nào nó không đủ để xác định duy nhất đáp án?
3. Trong không gian nhiều triệu chiều, vì sao điểm dừng gần như chắc chắn là yên ngựa chứ không phải cực tiểu địa phương?
4. Vì sao “chuẩn gradient nhỏ” là điều kiện dừng vô nghĩa khi huấn luyện với mini-batch?
5. Tăng batch size gấp bốn thì nhiễu gradient giảm bao nhiêu, và vì sao quan hệ đó khiến batch rất lớn kém hiệu quả về chi phí?
6. Loss giảm ổn định suốt 50 epoch nhưng mô hình vô dụng khi triển khai — ba nguyên nhân nào giải thích được chuyện đó, và mỗi nguyên nhân phân biệt bằng phép kiểm nào?

Nguồn

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [4] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [5] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [6] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.